

Supplementary Code: Bayesian Analysis of Restricted Mean Survival Time Adjusted for Covariates Using Pseudo-Observations

Léa Orsini

2026-08-12

Contents

1 Overview	1
1.1 Required files	1
1.2 Setup	2
2 Data generation (Scenario 5 of the simulation study)	2
3 Data Analysis with the Bayesian GMM	3
3.1 Unadjusted model (treatment effect only)	4
3.2 Adjusted model (with covariate adjustment)	6
4 Alternative: Adaptive Metropolis-within-Gibbs sampling via NIMBLE	8
Source and fit	8
Posterior summary	9
Trace plots	9
Comparison with Stan	10
5 Posterior probability of a clinically meaningful effect	11
6 Adapting this code to your own data	11

1 Overview

This document provides supplementary R code for the paper:

Orsini L., Lesaffre E., Yin G., Brard C., Dejardin D., Le Teuff G. *Bayesian analysis of restricted mean survival time adjusted for covariates using pseudo-observations*.

The code implements the **Bayesian Generalized Method of Moments (GMM)** for estimating the difference in Restricted Mean Survival Time (dRMST) between two treatment arms, with and without covariate adjustment. The workflow has two main steps:

1. **Compute pseudo-observations:** each individual's observed survival data (T_i, Δ_i) is transformed into a single pseudo-observation $y_{\tau,i}$ via the jackknife estimator (Equation 3 in the paper).
2. **Fit the Bayesian GMM:** the Stan model in `GMM_ind_single_tau.stan` samples from the posterior distribution of the regression coefficients using the No-U-Turn Sampler (NUTS), with a pseudo-likelihood that approximates the unknown full likelihood of the pseudo-observations (Equation 11 in the paper).

1.1 Required files

This document assumes that the following files are in the **same working directory** as this `.Rmd` file:

- `GMM_ind_single_tau.stan`: Stan model implementing the Bayesian GMM pseudo-likelihood
- `MCMC_traceplot.R`: helper function for MCMC trace plots with warm-up shading

1.2 Setup

```
library(tidyverse)    # data manipulation and plotting
library(pseudo)       # pseudo-observation computation (Pohar-Perme et al., 2017)
library(geepack)      # GEE (frequentist benchmark; not fitted here but included for reference)
library(rstan)        # R interface to Stan for Bayesian MCMC sampling

# Run MCMC chains in parallel using all available CPU cores
options(mc.cores = parallel::detectCores())

# Load helper function for trace plots
source("MCMC_traceplot.R")
```

2 Data generation (Scenario 5 of the simulation study)

This section reproduces **one replicate of Scenario 5** from Section 3 of the paper. Scenario 5 simulates a **2-arm RCT with a delayed treatment effect** (non-proportional hazards) and two prognostic covariates, $Z_1 \sim \mathcal{N}(0, 1)$ and $Z_2 \sim \text{Bernoulli}(0.5)$, whose effects on the time-to-event outcome satisfy the proportional hazards assumption. Two additional nuisance covariates (X_1, X_2) are also simulated, independent of the survival outcome, to allow assessment of model misspecification.

Event times are drawn from a Weibull distribution $S(t) = \exp\{-(\lambda t)^{1/\sigma}\}$, with arm-specific shape and scale parameters that produce delayed separation of the survival curves (they initially overlap before diverging). The true dRMST at $\tau = 5$ years is $\delta = 0.4911$ years. See Figure 2 and Table 1 of the Supplementary Materials for the full parameter table.

Weibull parameters by treatment arm:

	Control ($A = 0$)	Experimental ($A = 1$)
Shape σ	0.61	0.80
Scale λ	0.28	0.18

Censoring times are drawn from $\text{Uniform}(0, 11)$ with an additional administrative censoring at 8 years, yielding approximately 30% censoring overall.

```
set.seed(1)    # replicate s = 1; change to reproduce other replicates
n <- 200       # sample size (200 per arm is the main scenario in the paper)

# 1:1 treatment allocation: 1 = experimental arm, 0 = control arm
Trt <- rbinom(n, 1, 0.5)

# Arm-specific Weibull shape and scale parameters
shape <- 0.61 * (Trt == 0) + 0.8 * (Trt == 1)
scale <- 0.28 * (Trt == 0) + 0.18 * (Trt == 1)

# Prognostic covariates (associated with survival outcome)
Z1 <- rnorm(n, 0, 1)    # continuous covariate, log-HR = log(2)
Z2 <- rbinom(n, 1, 0.5) # binary covariate, log-HR = log(1.5)
```

```

b1 <- log(2)
b2 <- log(1.5)

# Nuisance covariates (not associated with survival: for misspecification tests)
X1 <- rnorm(n)
X2 <- rbinom(n, 1, 0.5)

# Simulate event times via the inverse CDF of the Weibull distribution,
# incorporating prognostic covariate effects through the hazard
U <- runif(n)
T_tilde <- (-log(U)) / ((scale)^(1 / shape) * exp(b1 * Z1 + b2 * Z2)))^shape

# Censoring: uniform up to 11 years, then administrative at 8 years
C <- runif(n, 0, 11)

time <- pmin(T_tilde, C, 8)
event <- as.numeric(time == T_tilde) # 1 = event observed, 0 = censored

simu <- data.frame(
  time = time,
  event = event,
  patID = 1:n,
  Trt = Trt,
  Z1 = Z1,
  Z2 = Z2,
  X1 = X1,
  X2 = X2
)

cat("Sample size:", n, "\n")

## Sample size: 200

cat("Events:", sum(event), "| Censored:", sum(event == 0),
    sprintf("(%.1f%%)\n", 100 * mean(event == 0)))

## Events: 123 | Censored: 77 (38.5%)

```

3 Data Analysis with the Bayesian GMM

Both models below follow the same two-step procedure:

Step 1 — Compute pseudo-observations using `pseudomean()` from the `pseudo` package. This implements the jackknife formula (Equation 3 in the paper):

$$y_{\tau,i} = n \int_0^\tau \hat{S}(t) dt - (n-1) \int_0^\tau \hat{S}^{-i}(t) dt$$

where $\hat{S}(t)$ is the Kaplan–Meier estimator and $\hat{S}^{-i}(t)$ is the Kaplan–Meier estimator leaving out individual i . The pseudo-observations are then used as the outcome variable in the regression.

Step 2 — Fit the Bayesian GMM using Stan. The pseudo-likelihood (Equation 11) is:

$$\tilde{L}(\beta \mid \mathbf{y}) \propto \exp\left\{-\frac{1}{2} U_n(\beta)^\top \Sigma_n^{-1}(\beta) U_n(\beta)\right\}$$

where $U_n(\beta)$ is the score vector and $\Sigma_n(\beta)$ is its empirical variance-covariance matrix. Weakly informative priors $\beta_p \sim \mathcal{N}(0, 10)$ are used for all parameters, as recommended by the Stan Development Team (2020).

The restriction time is $\tau = 5$ years. In the rare case where the last observed time in one arm is less than 5 years, `simu.tau` is adjusted to the minimum of the arm-specific maxima.

3.1 Unadjusted model (treatment effect only)

This model includes only an intercept and the treatment indicator:

$$\mu_i = \beta_0 + \delta \cdot A_i$$

- β_0 : RMST in the control arm
- δ (reported as `beta[2]`): **dRMST** the average gain in event-free time over 5 years attributed to the experimental treatment

```
tau <- 5    # restriction time in years (clinically chosen; 5 years is standard in oncology)
np <- 2    # number of parameters: intercept (beta_0) + treatment effect (delta)
n <- nrow(simu)

# Adjust tau if the maximum observed time in either arm is below 5 years
simu.tau <- min(tau,
               min(max(simu$time[simu$Trt == 1]),
                   max(simu$time[simu$Trt == 0])))

# Step 1: compute one pseudo-observation per individual
simu$rmst <- pseudomean(simu$time, simu$event, tmax = simu.tau)

# Step 2: build design matrix, first column is the intercept (all ones),
#         second column is the treatment indicator
X <- matrix(c(rep(1, n), simu$Trt), nrow = n, ncol = np)

data_unadj <- list(X = X, Y = simu$rmst, n = n, N = n, np = np)

# Compile the Stan model (only done once; subsequent runs use the cached binary)
GMM <- stan_model("GMM_ind_single_tau.stan")

# Fit: 3 chains x 1000 post-warmup iterations = 3000 posterior draws
# Chains run in parallel; total wall-clock time ~= time for 1 chain
fit_unadj <- sampling(
  GMM,
  data      = data_unadj,
  chains    = 3,
  iter      = 2000,    # total iterations per chain (warmup + sampling)
  warmup    = 1000,    # warm-up (discarded) iterations per chain
  seed      = 1,       # Stan MCMC seed for reproducibility
  init_r    = 1,       # initial values drawn from Uniform(-1, 1)
  cores     = 3,
  save_warmup = TRUE,   # retain warm-up draws (required for trace plots)
  refresh   = 1000     # print progress every 1000 iterations
)
```

Posterior summary

The table reports the posterior mean (`mean`), standard deviation (`sd`), 2.5th and 97.5th percentiles (95% equal-tailed credible interval), the Monte Carlo standard error (`se_mean`), the Gelman–Rubin convergence diagnostic (Rhat close to 1.00 indicates convergence), and the effective sample size (`n_eff`).

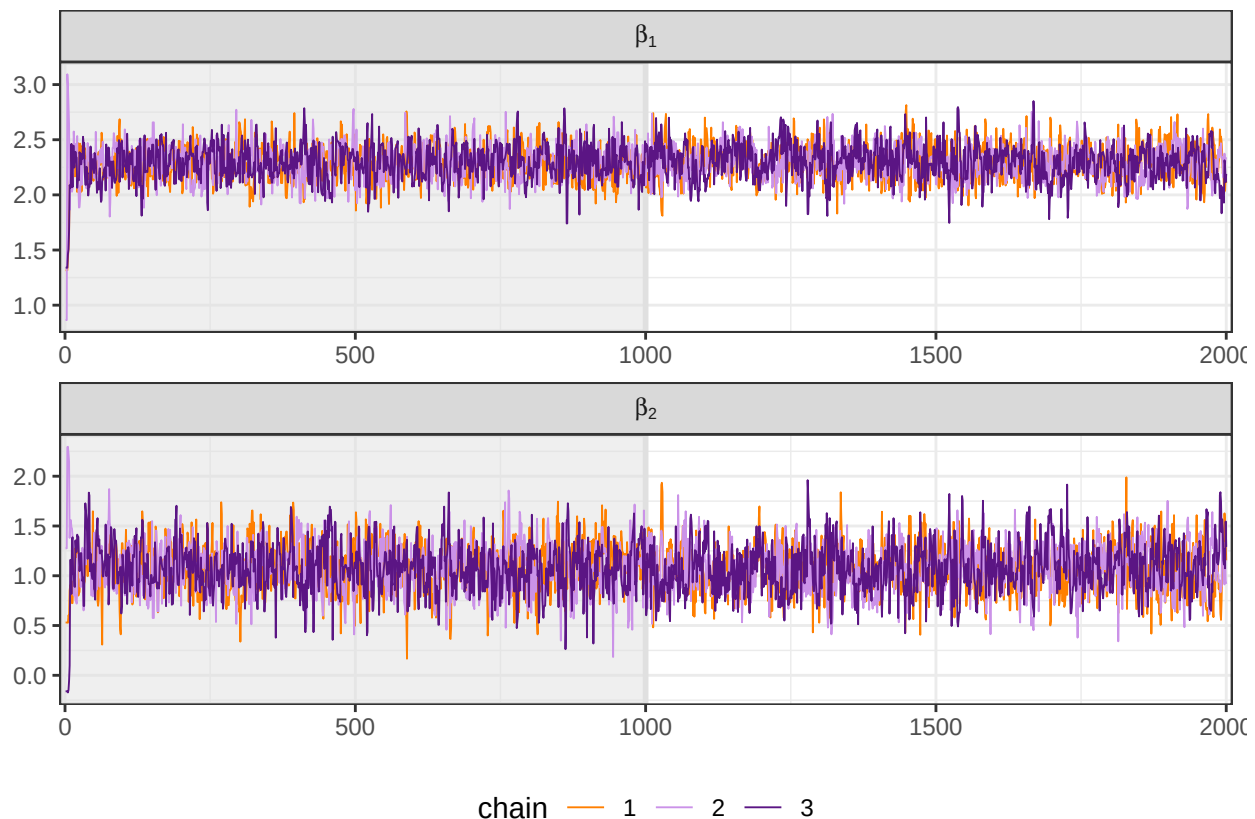
```
unadj_summ <- summary(fit_unadj, pars = c("beta[1]", "beta[2]"))$summary
rownames(unadj_summ) <- c(
  "beta[1] Intercept (RMST in control arm)",
  "beta[2] delta      (dRMST: treatment effect)"
)
round(unadj_summ, 3)
```

##			mean	se_mean	sd	2.5%	25%
##	beta[1]	Intercept (RMST in control arm)	2.297	0.005	0.158	1.994	2.188
##	beta[2]	delta (dRMST: treatment effect)	1.075	0.007	0.241	0.617	0.913
##			50%	75%	97.5%	n_eff	Rhat
##	beta[1]	Intercept (RMST in control arm)	2.296	2.404	2.610	992.221	1.003
##	beta[2]	delta (dRMST: treatment effect)	1.071	1.241	1.547	1136.043	1.003

MCMC trace plots

Each panel shows the sampled values of a parameter over MCMC iterations for each of the three chains (coloured differently). The shaded region on the left is the warm-up phase. Well-mixing chains (no trends, no divergence, good overlap between chains) indicate that the posterior has been adequately explored.

```
MCMC_traceplot(fit_unadj, c("beta[1]", "beta[2]"))
```



3.2 Adjusted model (with covariate adjustment)

This model adds the two prognostic covariates Z_1 and Z_2 to the regression:

$$\mu_i = \beta_0 + \delta \cdot A_i + \beta_1 Z_{1i} + \beta_2 Z_{2i}$$

Including prognostic covariates reduces residual variance in the pseudo-observations and typically yields a more precise estimate of δ (narrower credible interval). According to the simulation results in Table 2 of the paper, the gain in precision from the unadjusted to the adjusted model is approximately 8% for Scenario 5.

```
np <- 4 # intercept + treatment + Z1 + Z2
n <- nrow(simu)

simu.tau <- min(tau,
               min(max(simu$time[simu$Trt == 1]),
                   max(simu$time[simu$Trt == 0])))

# Pseudo-observations are the same as before (same tau, same data)
simu$rmst <- pseudomean(simu$time, simu$event, tmax = simu.tau)

# Design matrix: intercept | treatment | Z1 | Z2
X <- matrix(c(rep(1, n), simu$Trt, simu$Z1, simu$Z2), nrow = n, ncol = np)

data_adj <- list(X = X, Y = simu$rmst, n = n, N = n, np = np)

# Stan model is already compiled above; re-use it
GMM <- stan_model("GMM_ind_single_tau.stan")

fit_adj <- sampling(
  GMM,
  data      = data_adj,
  chains    = 3,
  iter      = 2000,
  warmup    = 1000,
  seed      = 1,
  init_r    = 1,
  cores     = 3,
  save_warmup = TRUE,
  refresh   = 1000
)
```

Posterior summary

```
adj_summ <- summary(
  fit_adj,
  pars = c("beta[1]", "beta[2]", "beta[3]", "beta[4]")
)$summary
rownames(adj_summ) <- c(
  "beta[1] Intercept (RMST in control arm, Z1=0, Z2=0)",
  "beta[2] delta (dRMST: treatment effect, adjusted)",
  "beta[3] beta_1 (dRMST per unit increase in Z1)",
  "beta[4] beta_2 (dRMST: Z2=1 vs Z2=0)"
)
round(adj_summ, 3)
```

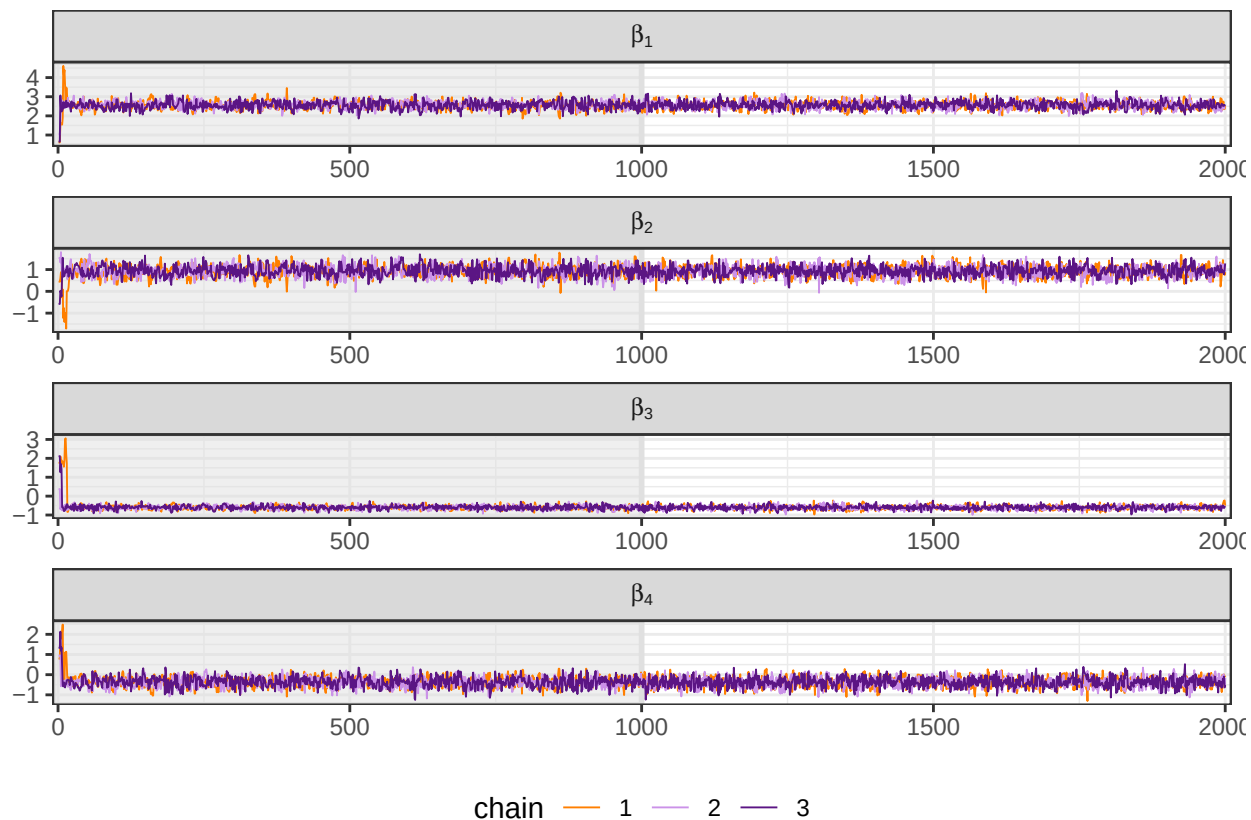
```
##
## beta[1] Intercept (RMST in control arm, Z1=0, Z2=0) 2.569 0.005 0.202
## beta[2] delta (dRMST: treatment effect, adjusted) 0.927 0.006 0.246
## beta[3] beta_1 (dRMST per unit increase in Z1) -0.604 0.002 0.107
## beta[4] beta_2 (dRMST: Z2=1 vs Z2=0) -0.375 0.006 0.245
##
## 2.5% 25% 50%
## beta[1] Intercept (RMST in control arm, Z1=0, Z2=0) 2.179 2.433 2.563
## beta[2] delta (dRMST: treatment effect, adjusted) 0.439 0.766 0.931
## beta[3] beta_1 (dRMST per unit increase in Z1) -0.816 -0.672 -0.604
## beta[4] beta_2 (dRMST: Z2=1 vs Z2=0) -0.864 -0.543 -0.374
##
## 75% 97.5% n_eff
## beta[1] Intercept (RMST in control arm, Z1=0, Z2=0) 2.703 2.970 1525.145
## beta[2] delta (dRMST: treatment effect, adjusted) 1.095 1.398 1774.112
## beta[3] beta_1 (dRMST per unit increase in Z1) -0.537 -0.383 2051.043
## beta[4] beta_2 (dRMST: Z2=1 vs Z2=0) -0.209 0.091 1879.094
##
## Rhat
## beta[1] Intercept (RMST in control arm, Z1=0, Z2=0) 1.000
## beta[2] delta (dRMST: treatment effect, adjusted) 1.000
## beta[3] beta_1 (dRMST per unit increase in Z1) 1.001
## beta[4] beta_2 (dRMST: Z2=1 vs Z2=0) 0.999
```

Interpretation:

- β_0 : estimated RMST in the control arm when $Z_1 = 0$ and $Z_2 = 0$
- $\hat{\delta}$: estimated dRMST between arms, adjusted for the prognostic covariates; compare to β_2 in the unadjusted model, precision should increase (smaller sd)
- $\hat{\beta}_1$: change in RMST associated with a one-unit increase in Z_1
- $\hat{\beta}_2$: dRMST between individuals with $Z_2 = 1$ vs $Z_2 = 0$

MCMC trace plots

```
MCMC_traceplot(fit_adj, c("beta[1]", "beta[2]", "beta[3]", "beta[4]"))
```



4 Alternative: Adaptive Metropolis-within-Gibbs sampling via NIMBLE

As noted in Section 2.3 of the paper, the Bayesian GMM is **sampler-agnostic**: any MCMC algorithm can be used to draw from the posterior defined by the pseudo-likelihood (Equation 12). The `GMM_ind_single_tau.R` file provides an alternative implementation using the **adaptive Metropolis-within-Gibbs algorithm** from the NIMBLE package (de Valpine et al., 2017), instead of the No-U-Turn Sampler (NUTS) used in Stan above.

This section fits the same unadjusted model (intercept + treatment) as Section 3.1 and compares the posterior summaries to verify that both samplers converge to the same posterior.

Note on extending to more covariates: the initial values inside `GMM_ind_single_tau.R` are hardcoded for a two-parameter model (`beta = c(1, 1)`). To fit the adjusted model with additional covariates, update the `ModelInits` list in that file to match `np`, for example, `beta = c(1, 1, 0, 0)` for four parameters.

Package dependencies: `GMM_ind_single_tau.R` requires the `nimble` and `ggpubr` packages to be installed.

Source and fit

```
library(nimble)
source("GMM_ind_single_tau.R") # defines GMM_ind_MCMC_fit()

# Re-use the unadjusted design matrix and pseudo-observations from Section 3.1
```



```

data_nimble <- list(
  Y = simu$rmst,
  X = data_unadj$X, # intercept + treatment (2 columns)
  n = n,
  np = 2
)

# 2 chains, 4000 iterations each; burn-in = 2000, thinning by 2
# + 1000 posterior draws per chain = 2000 total
fit_nimble <- GMM_ind_MCMC_fit(
  data = data_nimble,
  parM = list(niter = 4000, burnin = 2000, thin = 2)
)

```

Posterior summary

Post-burn-in, thinned samples are stored in `fit_nimble$chain`. NIMBLE converts parameter names such as `beta[1]` to `beta.1`. when constructing R data frames, so the column names differ from Stan output.

```

post <- fit_nimble$chain
beta_cols <- grep("^beta", names(post), value = TRUE)

nimble_summ <- t(apply(beta_cols, function(col) {
  x <- post[[col]]
  c(mean = round(mean(x), 3),
    sd = round(sd(x), 3),
    `2.5%` = round(quantile(x, 0.025), 3),
    `97.5%` = round(quantile(x, 0.975), 3))
}))
rownames(nimble_summ) <- c(
  "beta[1] Intercept (RMST in control arm)",
  "beta[2] delta (dRMST: treatment effect)"
)
nimble_summ

```

##		mean	sd	2.5%.2.5%	97.5%.97.5%
## beta[1]	Intercept (RMST in control arm)	2.312	0.160	2.005	2.628
## beta[2]	delta (dRMST: treatment effect)	1.066	0.259	0.531	1.569

Trace plots

Each panel shows sampled values across post-burn-in iterations for the two chains. As with the Stan trace plots, well-mixing chains with no trends confirm adequate convergence.

```

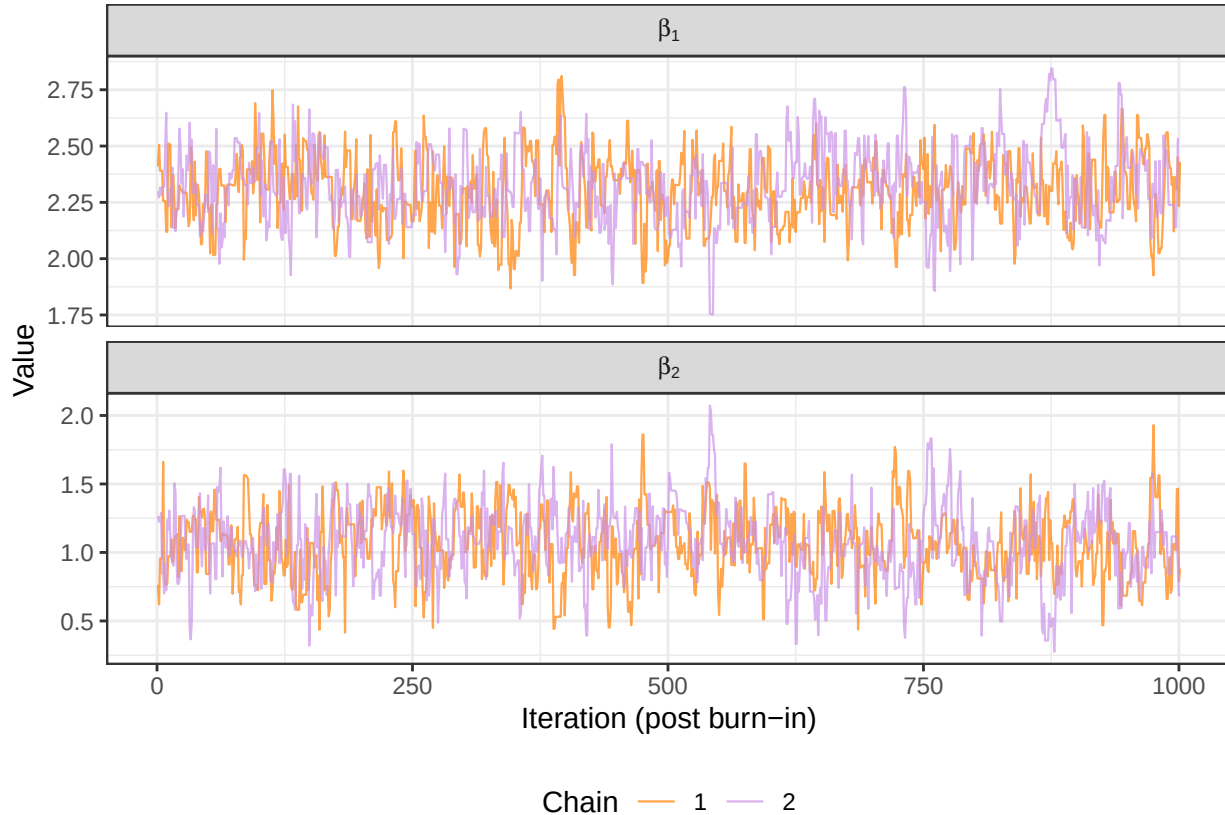
post |>
  select(all_of(c(beta_cols, "chain", "sampleid"))) |>
  pivot_longer(cols = all_of(beta_cols),
    names_to = "parameter", values_to = "value") |>
  mutate(parameter = case_when(
    parameter == "beta.1." ~ "beta[1]",
    parameter == "beta.2." ~ "beta[2]"
  )) |>
  ggplot(aes(x = sampleid, y = value, colour = factor(chain))) +
  geom_line(alpha = 0.7, linewidth = 0.4) +
  facet_wrap(~parameter, scales = "free_y", ncol = 1,

```

```

labeller = label_parsed) +
labs(x = "Iteration (post burn-in)", y = "Value", colour = "Chain") +
scale_colour_manual(values = c("#FF7F00", "#CB92E8")) +
theme_bw() +
theme(legend.position = "bottom")

```



Comparison with Stan

Both samplers target the same posterior. The table below confirms that the posterior means and standard deviations are consistent between NIMBLE and Stan, as expected from a sampler-agnostic approach.

```

stan_summ  <- summary(fit_unadj, pars = c("beta[1]", "beta[2]"))$summary[, c("mean", "sd")]
nimble_means <- sapply(beta_cols, function(col) c(mean = mean(post[[col]]), sd = sd(post[[col]])))

comparison <- data.frame(
  Parameter = c("beta[1] Intercept", "beta[2] delta (dRMST)",
  Stan_mean = round(stan_summ[, "mean"], 3),
  Stan_sd   = round(stan_summ[, "sd"], 3),
  NIMBLE_mean = round(nimble_means["mean", ], 3),
  NIMBLE_sd   = round(nimble_means["sd", ], 3)
)
rownames(comparison) <- NULL
comparison

```

##	Parameter	Stan_mean	Stan_sd	NIMBLE_mean	NIMBLE_sd
## 1	beta[1] Intercept	2.297	0.158	2.312	0.160
## 2	beta[2] delta (dRMST)	1.075	0.241	1.066	0.259

5 Posterior probability of a clinically meaningful effect

A distinctive advantage of the Bayesian framework highlighted in Section 4 of the paper is the ability to compute the probability that any parameter exceeds a clinically meaningful threshold, directly from the MCMC draws, without additional approximation.

This is illustrated in the paper using the Getug-AFU 15 trial: the estimated posterior probability of the dRMST between the two treatment arms over 5 years to be higher than 3 months is $P(\delta \geq 3 \text{ months}) = 0.973$ for the covariate-adjusted model. The code below demonstrates the same calculation on the simulated data.

```
# Extract all post-warmup draws for delta (treatment effect) from the adjusted model
posterior_delta <- rstan::extract(fit_adj, pars = "beta[2]", permuted = TRUE)$`beta[2]`

# Clinically chosen threshold: is the treatment beneficial by at least 3 months?
threshold <- 3 / 12 # 3 months expressed in years (same unit as RMST)

prob <- mean(posterior_delta > threshold)
mc_se <- sd(posterior_delta > threshold) / sqrt(length(posterior_delta))

cat(sprintf("Posterior probability P(delta > %.0f months) = %.3f (MC-SE = %.4f)\n",
            threshold * 12, prob, mc_se))
```

```
## Posterior probability P(delta > 3 months) = 0.997 (MC-SE = 0.0010)
```

```
# The same calculation can be applied to any covariate effect.
```

```
# For example, the probability that Z2 shortens RMST by more than 0.5 years:
```

```
posterior_beta4 <- rstan::extract(fit_adj, pars = "beta[4]", permuted = TRUE)$`beta[4]`
prob_z2 <- mean(posterior_beta4 < -0.5)
cat(sprintf("Posterior probability P(beta_2 < -0.5 years) = %.3f\n", prob_z2))
```

```
## Posterior probability P(beta_2 < -0.5 years) = 0.302
```

These probabilities are obtained by simple arithmetic on the MCMC sample, no closed-form expression is required.

6 Adapting this code to your own data

To apply the Bayesian GMM to a real or different simulated dataset, replace the `simu` data frame and follow the same steps. The Stan file `GMM_ind_single_tau.stan` is fully general and does not need to be modified.

1. **Prepare your data:** ensure you have a time variable, a binary event indicator (1 = event, 0 = censored), a treatment variable, and any adjustment covariates.
2. **Choose tau:** the restriction time should be chosen on clinical grounds before data analysis (e.g., 5 years for an oncology trial). If the last observed time in any arm is shorter than `tau`, use `simu.tau` as shown above.
3. **Compute pseudo-observations:** `pseudomean(time, event, tmax = tau)` returns one pseudo-observation per row.
4. **Build the design matrix X:** always include a column of ones (intercept) first, followed by the treatment variable, then any covariates. The number of columns is `np`.
5. **Fit and interpret:** run `sampling()` as above, then use `summary(fit, pars = ...)` for posterior summaries and `rstan::extract()` to compute posterior probabilities.

Prior sensitivity: the default prior $\beta_p \sim \mathcal{N}(0, 10)$ is weakly informative. For small samples ($n \leq 50$) with many covariates, consider a tighter prior such as $\mathcal{N}(0, 1)$ to reduce posterior variance (see Section 3.4 of the paper).
